



并行计算

12 分布式系统应用

中国人民大学信息学院 · 鄂金龙



本课目录

- 分布式存储系统
- 分布式计算服务
- 分布式通信框架





分布式存储系统

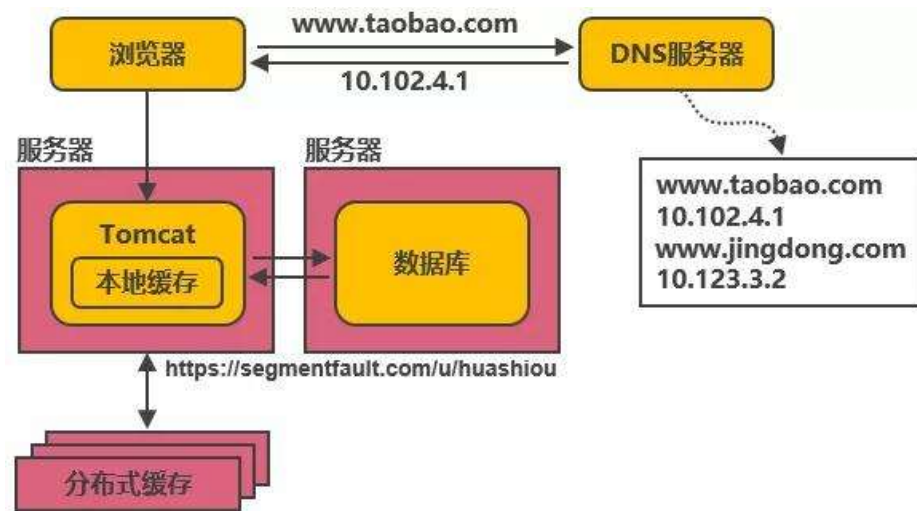
• GFS

– 现代分布式系统的开始

- Google文件系统，针对Google公司**实际应用问题**：廉价设备、GB级文件、多客户高并发追加 (Append) 操作
- 抽象对外屏蔽，用户不用管分布式的细节 (*Big Storage*)
- 伴随松散的分布式互联网后台架构 (*Big Websites*)

– vs 先前分布式文件系统

- 高并发、高负载业务驱动
- 性能优先、低成本、规模超大
- 简单、实用（扩展性/容错性好）



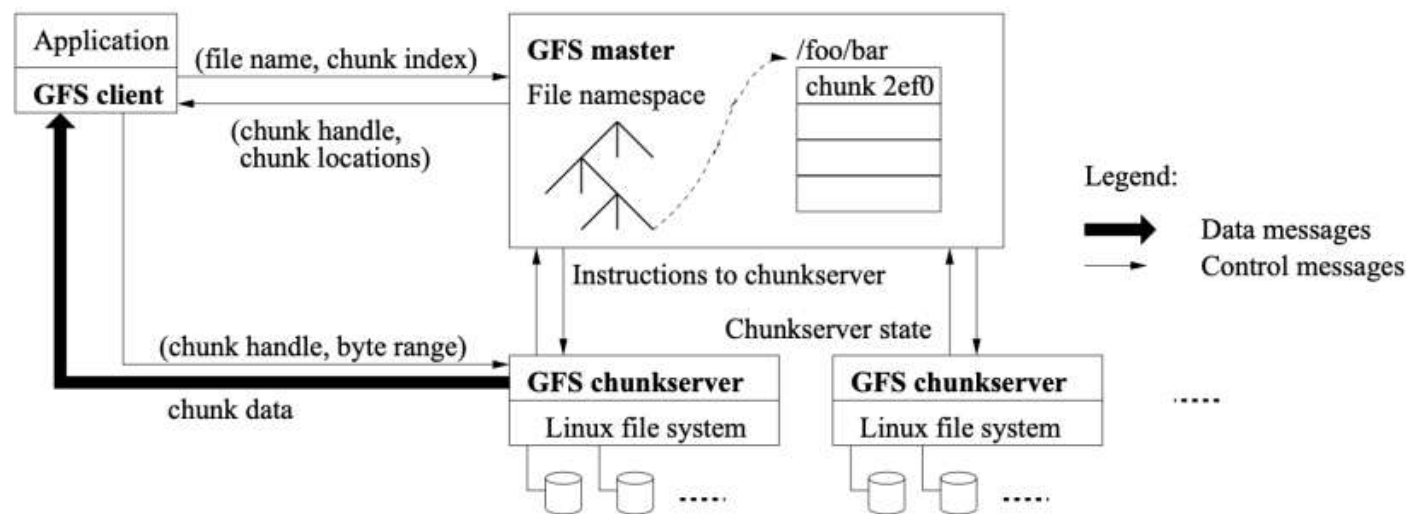


分布式存储系统

• GFS

– 系统架构

- 多个客户端 (Client)：不支持POSIX接口，而是单独的访问API (create, delete, open, close, read, write, *snapshot*, *record append*)
- 单个主控服务器 (Master)：文件名 -> 块地址 (块服务器)，*不处理数据流*
- 多个块服务器 (Chunkserver)：响应块查询返回块数据，*多副本放置*



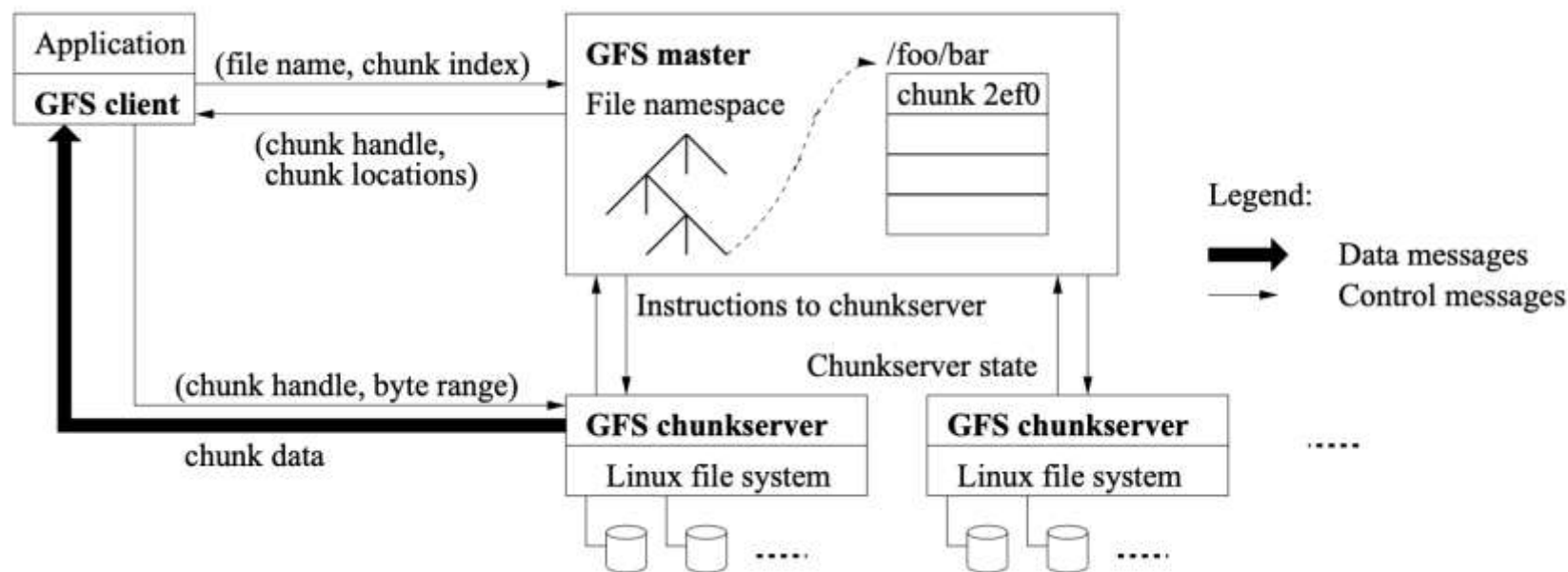


分布式存储系统

• GFS

– 读流程

- 客户端先联系主控服务器**定位**到块服务器（客户端会有**块地址缓存**，但有效期有限）
- 客户端直接访问块服务器获取数据（不一定是**主副本**，任意一个均可，一般是网络距离更近的）



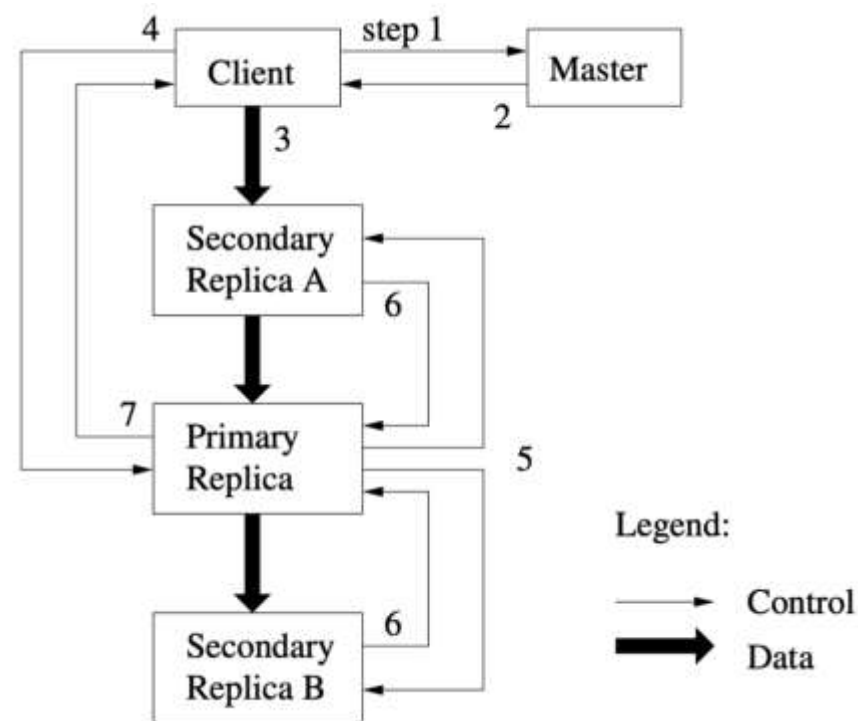


分布式存储系统

• GFS

– 写流程

- 客户端需要同步更新所有副本才能完成写操作
- (1~2) 客户端从主控服务器获取块副本位置和租约
- (3) 客户端将写入数据发到相关块服务器缓存
- (4) 客户端向主副本所在块服务器发写请求
- (5~6) 主块服务器排定修改顺序，控制写入辅助副本
- (7) 主块服务器将信息反馈客户端，出错则重复(3~7)
- 每次写完成后块版本号+1 -> 用于检测过时的块副本





分布式存储系统

• GFS

– 控制设计

- **元数据**: 文件和块的命名空间、文件到块的映射信息、块副本的位置信息 (不多于64B, 块大小64MB)
- **操作日志**: 唯一持久化, 全局有序以解决并发冲突, 基于检查点机制 `/d1/d2/.../dn/leaf`
- **命名空间**: 管理完整路径名到元数据的查找表, 通过前缀压缩放在内存中
- **心跳消息**: 主控服务器与各块服务器周期性通信 -> 给出指令 & 收集状态
- **垃圾回收**: 定期扫描元数据, 延迟回收过时数据 (版本号检测停机导致的过期副本)

* Sanjay Ghemawat, Howard Gobioff, and Shun-Tak Leung, "The Google File System", SOSP 2003



分布式存储系统

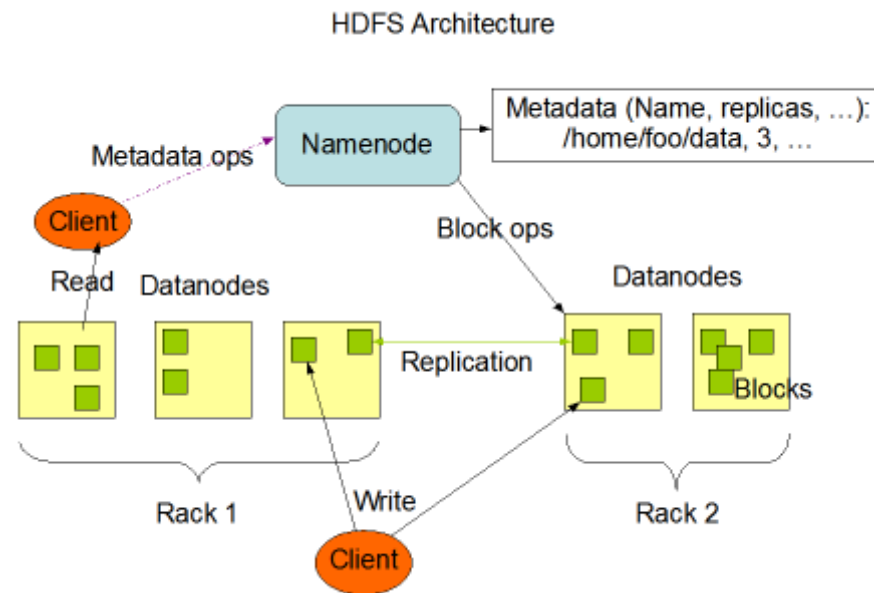
• HDFS

– 简介

- Hadoop分布式文件系统，主要由Yahoo!开发的开源系统，面向流数据模式访问和处理超大文件的需求开发，因具有高容错性而可以部署在低廉的硬件上

– 系统架构 (主从模式)

- 名称节点 (Namenode): 管理文件系统命名空间、客户端对文件的访问操作、数据块与数据节点的映射
- 数据节点 (Datanode): 管理文件分成的数据块存放在若干节点、处理客户端的文件读写请求



Java语言开发, 商用计算机部署集群



Ceph: 支持块存储、对象存储和文件存储 (统一存储), 支持POSIX接口

* Konstantin Shvachko, Hairong Kuang, Sanjay Radia, Robert Chansler, "The Hadoop Distributed File System", MSST 2010





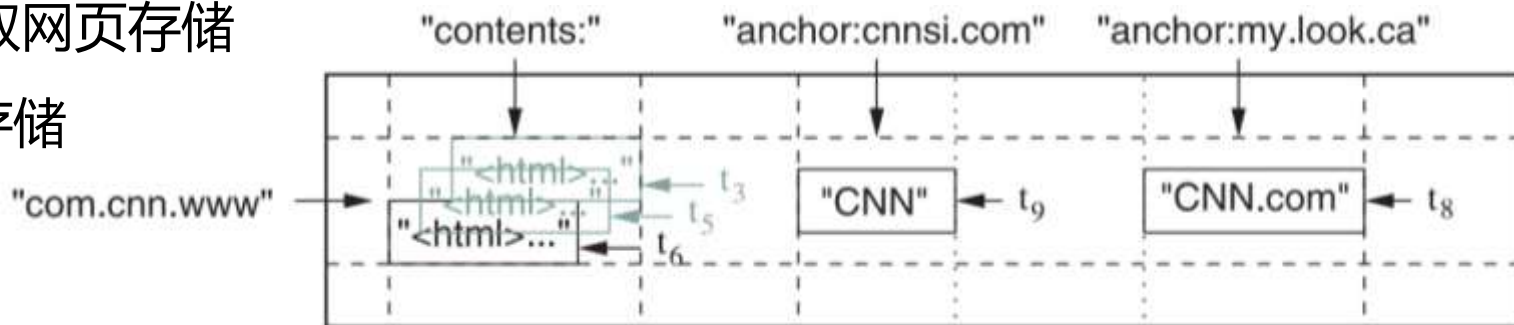
• Bigtable

– 简介

- Google开发的管理大规模结构化数据 (PB级) 的分布式KV数据库, 开启NoSQL
- 应用: 从后端大批量处理到实时在线服务, 包括网页索引、Google分析、Google金融、Orkut、Google Earth、Writely (Google Docs) 等

– 数据模型

- 稀疏的多维排序映射表: (row: string, column: string, time: int64) -> string
- 支持时间戳多版本 -> 爬取网页存储
- 底层实际使用key-value存储
- 数据内容主要是字符串





分布式存储系统

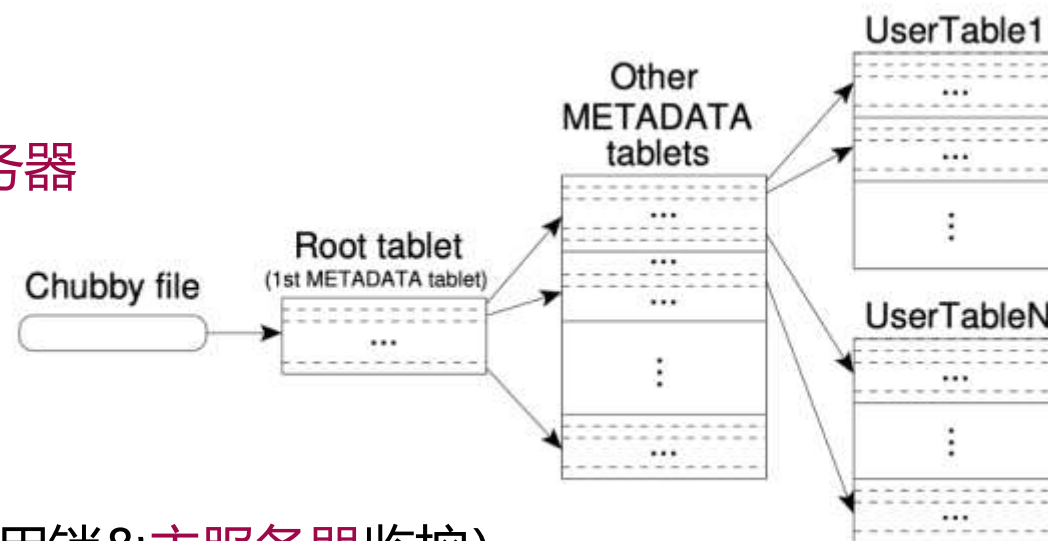
• Bigtable

– 系统依赖框架

- 利用GFS做底层分布式文件/日志存储，负责数据复制 -> 保证可靠性/可用性
- 利用Chubby保证多副本一致性，存放模式 (表的列簇) 信息和访问控制列表

– 表块 (Tablet) 分配和定位

- 主服务器将表块 (数据分片) 分配到表块服务器
- 类似B+树的三层结构存放表块位置信息
- 每个表块服务器管理10-1000个表块
- 用户直接与表块服务器交互读写表块
- Chubby目录创建文件跟踪表块服务器 (专用锁&主服务器监控)





分布式存储系统

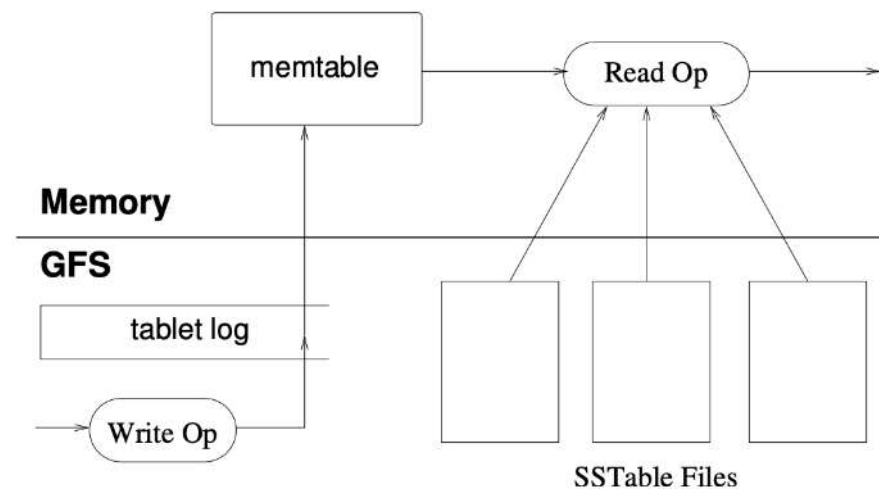
• Bigtable

– 表块服务

- 表块永久存放在GFS中，更新记录到提交日志中，最近提交的缓存在**memtable**
- 存储格式**SSTable**，包含序列**数据块** (默认64KB)，**索引**提供从关键字到值的映射
- 写性能好，读性能较差（相对RocksDB等LSM-Tree存储引擎）



Spanner: Google开发的**全球分布式**、多版本存储的KV数据库
(*Bigtable的扩展版*)



* Fay Chang, Jeffrey Dean, Sanjay Ghemawat, and et al., "Bigtable: A Distributed Storage System for Structured Data", OSDI 2006





分布式存储系统



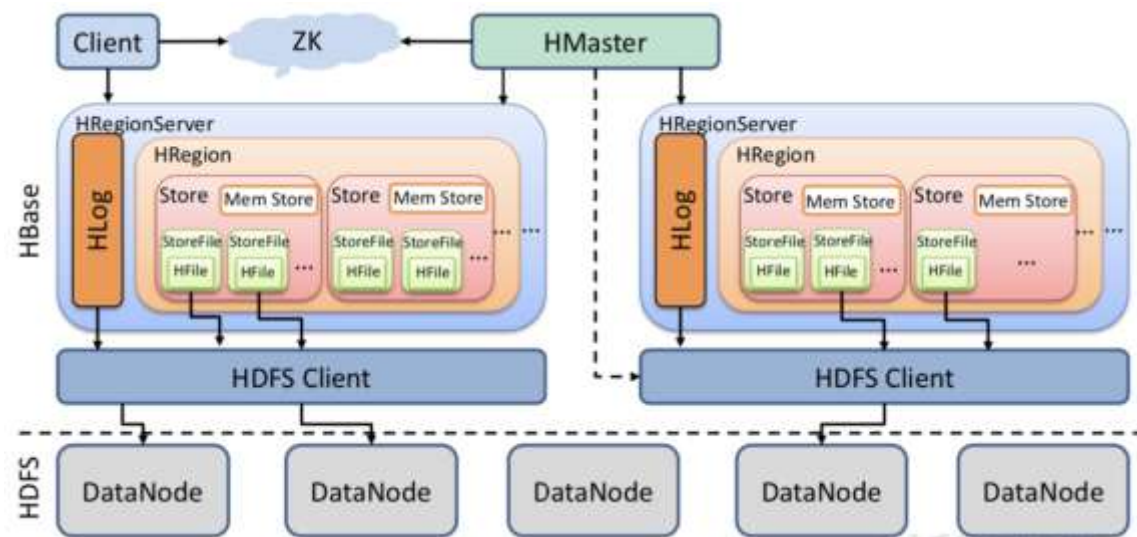
• HBase

– 简介

- Apache Hadoop的开源分布式数据库，基于Bigtable开发，面向列的存储模型
- 存储松散型数据（介于KV和关系型数据库之间），底层依赖于HDFS和ZooKeeper

– 系统架构（主从模式）

- HRegion服务器
 - HRegion基于表主键范围分片Region（列簇Store-> MemStore & Hfile）
 - HLOG存储数据日志（先写模式）
- HMaster服务器：存储数据分片映射





分布式存储系统

• Dynamo

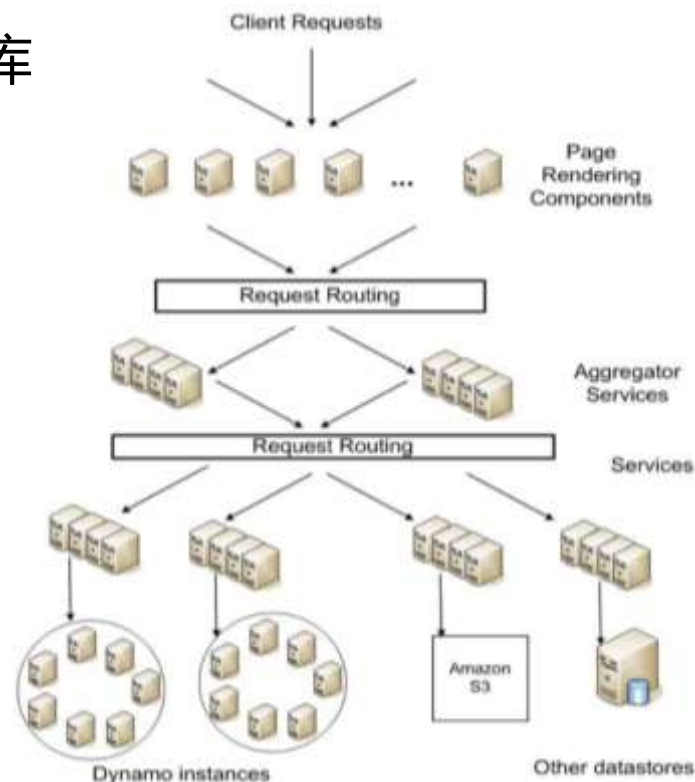


– 简介

- Amazon开发的高可用分布式KV数据存储，管理松散服务器运行电商平台
- Cassandra、Voldemort是Dynamo风格的开源分布式数据库

– 用户访问流程

- Dynamo的节点是P2P结构，任何一个节点均可以服务用户
- 负载均衡器将用户请求 (get/put(key, ...)) 发送给目标key的协调节点 (键的优先列表中第一个无故障节点)
- 此后由协调节点负责，按照Quorum的 $R+W > N$ 原则进行读/写访问



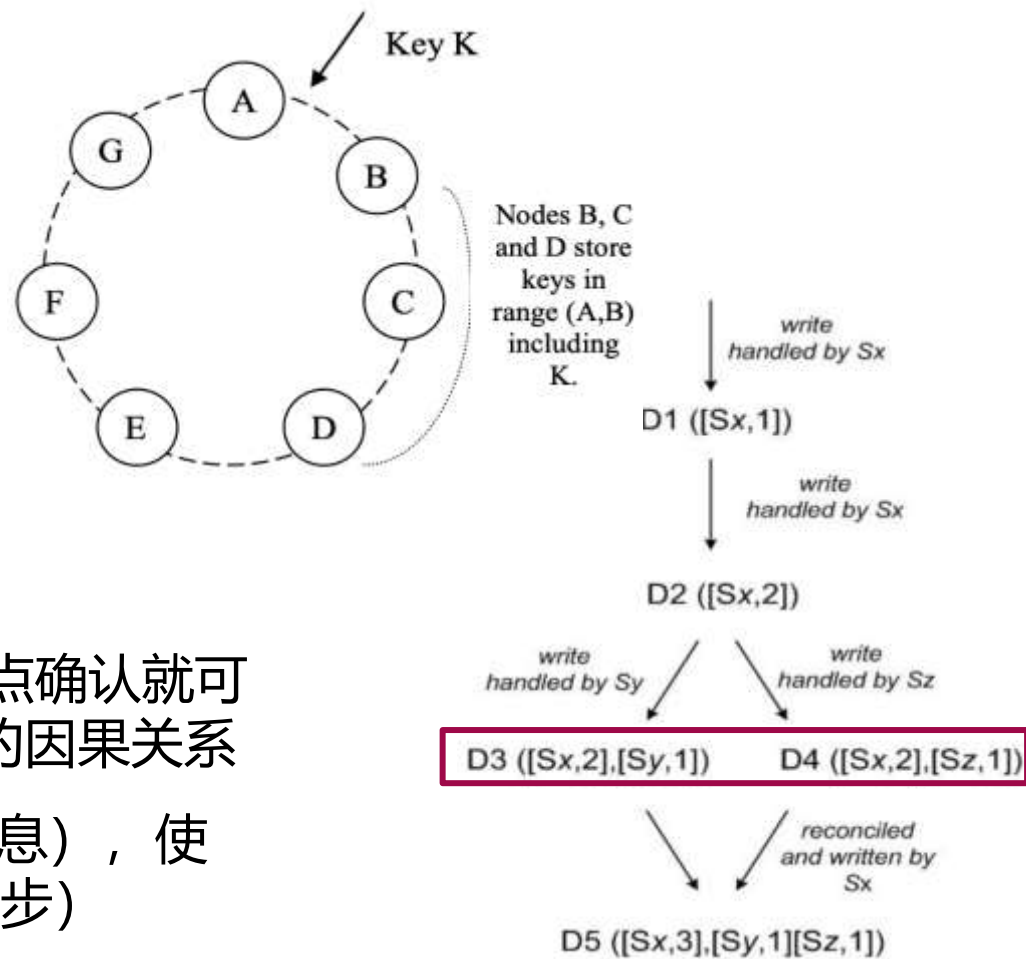


分布式存储系统

• Dynamo

– 数据分片和复制

- 满足规模可扩展性 -> 一致性哈希算法动态确定存取数据的节点，映射到环上的虚拟节点
- 该虚拟节点（映射到物理上的协调节点）本地和顺时针后续N-1个虚拟节点都存一个副本



– 版本更新和故障处理

- $R + W > N$ 不能保证副本同步更新，未经过多数节点确认就可以写操作覆盖 -> 基于向量时钟找出不同版本间的因果关系
- 通过Gossip协议检测可达节点（避免存储节点信息），使用Merkle树的同步复制来处理故障（后台快速同步）

* Giuseppe DeCandia, Deniz Hastorun, et al, "Dynamo: Amazon's Highly Available Key-value Store", SOSP 2007





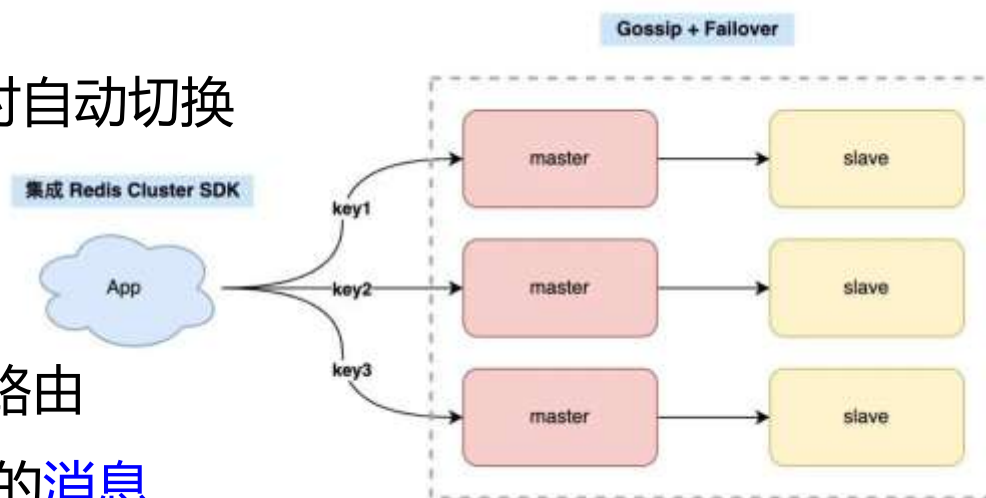
• Redis

– 简介

- 远程字典服务，开源的分布式非关系型 (KV) 数据库 (2009 Merzia->Pivotal)
- 特点：运行在内存 (读写性能高) & 可持久化到磁盘、对复杂数据结构 (list, set, zset, hash) 提供原子操作、支持主从模式数据备份、支持发布/订阅

– 系统架构

- Redis集群：节点间Gossip协议探测状态，故障时自动切换
- 客户端通过SDK集成，基于键查找节点读写
- 自动适配节点增加和删除，业务侧无感知
- 适配早期业务侧：一组代理负责客户端数据转发路由
- Redis v5.0新增Stream数据结构，用于消息队列的消息





分布式计算服务

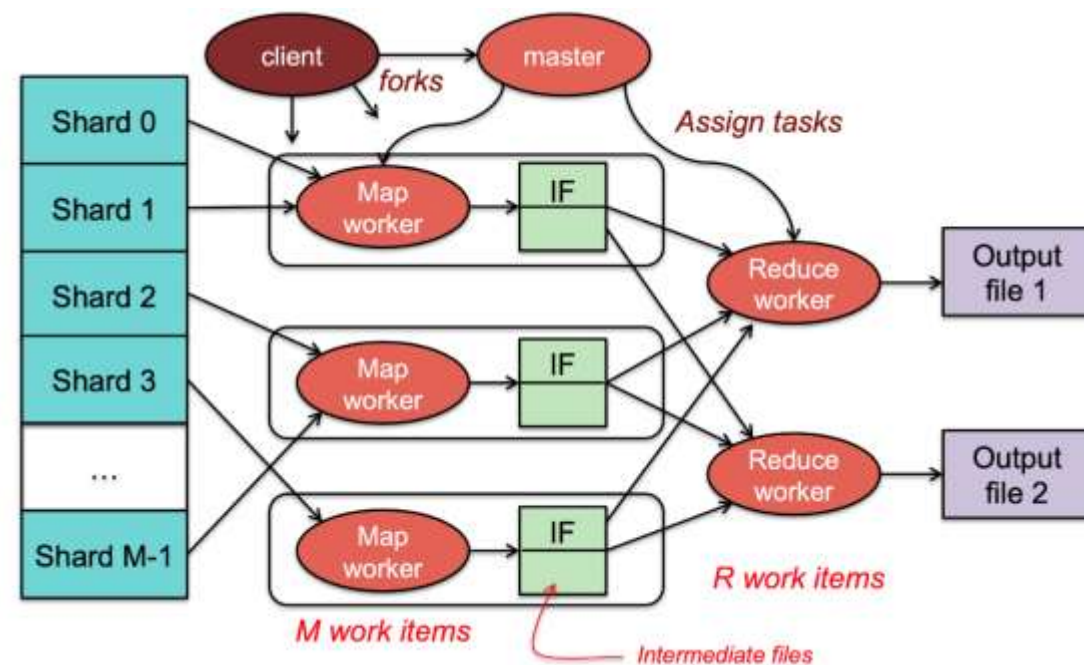
• MapReduce

– 简介

- 一种用于大规模数据集（一般大于1TB）的并行运算编程模型（大数据处理框架）
- 应用场景：Google网页处理建立索引，Hadoop大规模企业数据处理和分析

– 模型框架

- **Map函数**：从数据源（如HDFS）获取键值对<key, value>集合 -> 拆分数据碎片输入计算，生成一个中间<key, value>集合
- **Reduce函数**：接收一个中间key和所有相关value集合 -> 将合并结果生成一个新的输出<key, value>集合





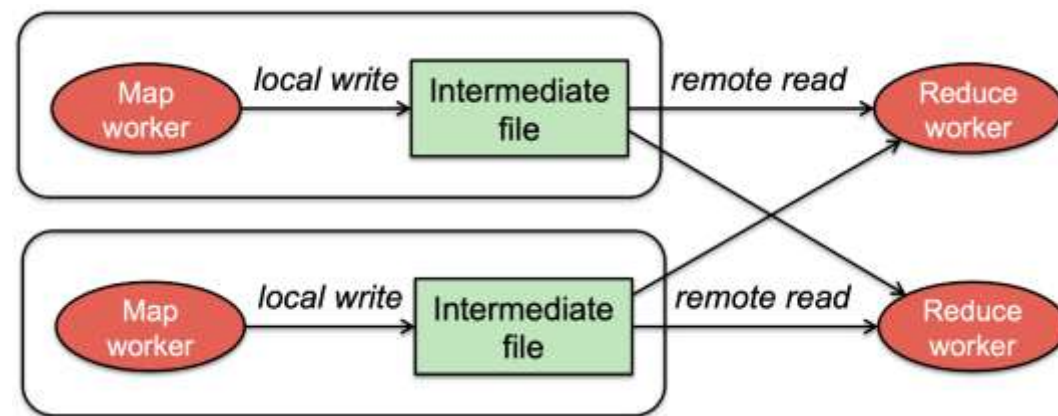
• MapReduce

– 执行步骤

- 拆分：将输入数据分成M个分片，一般大小为64MB
- 派生 (Forking)：管理者向空闲工作者分配Map任务或Reduce任务
- Map：将分片解析出键值对，输入函数计算中间键值对，周期性写入中间文件
- 分区：将任务划分给R个工作者，将Map结果计算键的哈希值和它们匹配
- 洗牌 (Shuffling)：读取中间文件数据，按键排序并分组相同键的数据
- Reduce：将key和对应的一组中间值输入函数，计算后追加到输出文件

$\text{hash}(\text{key}) \bmod R$

$\langle \text{key}, (\text{value1}, \text{value2}, \text{value3}, \text{value4}, \dots) \rangle$





• MapReduce

– 典型实例：单词计数

Map(String key, String value):

for each word w in value:

EmitIntermediate(w, '1');

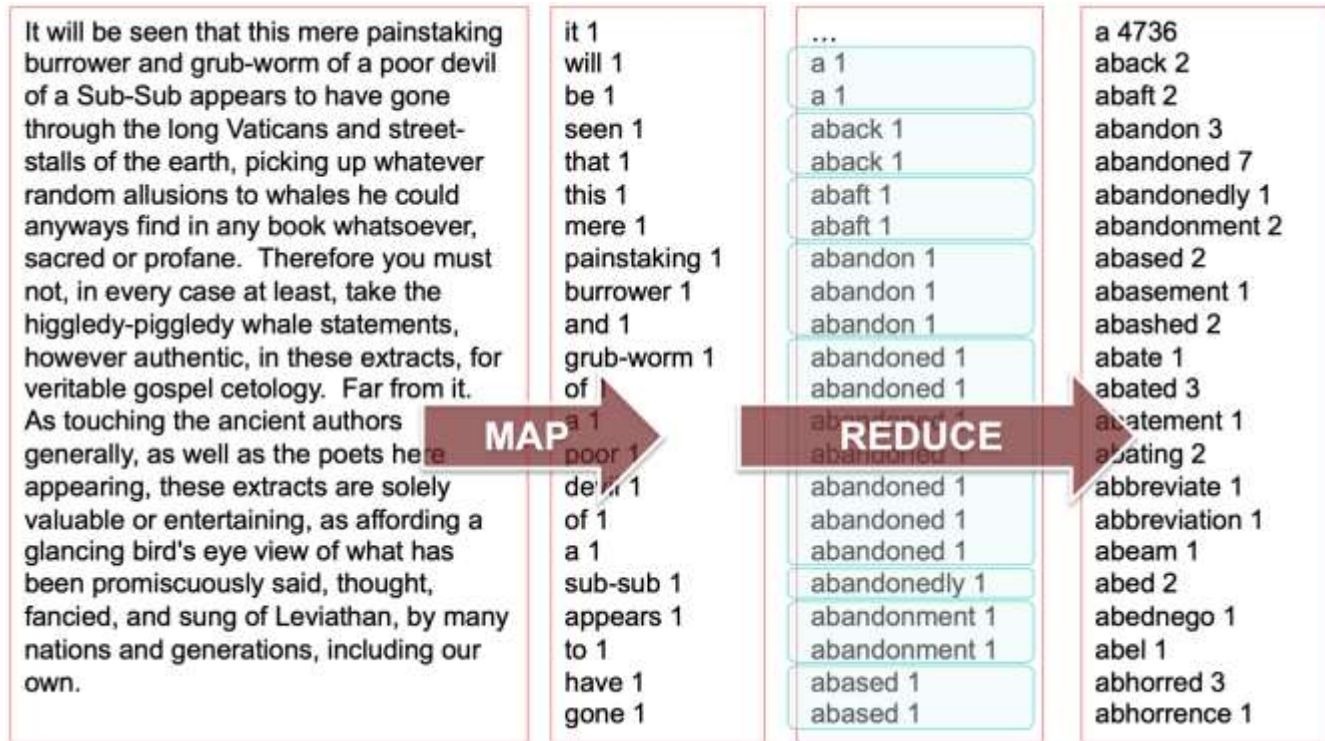
Reduce(String key, Iterator values):

int result = 0;

for each v in values:

Result += ParseInt(v);

Emit(AsString(result));



输入参数	Map函数	Reduce函数
key	文档名称	一个单词
value	文档内容	计数列表

* Jeffrey Dean and Sanjay Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters", OSDI 2004





分布式计算服务

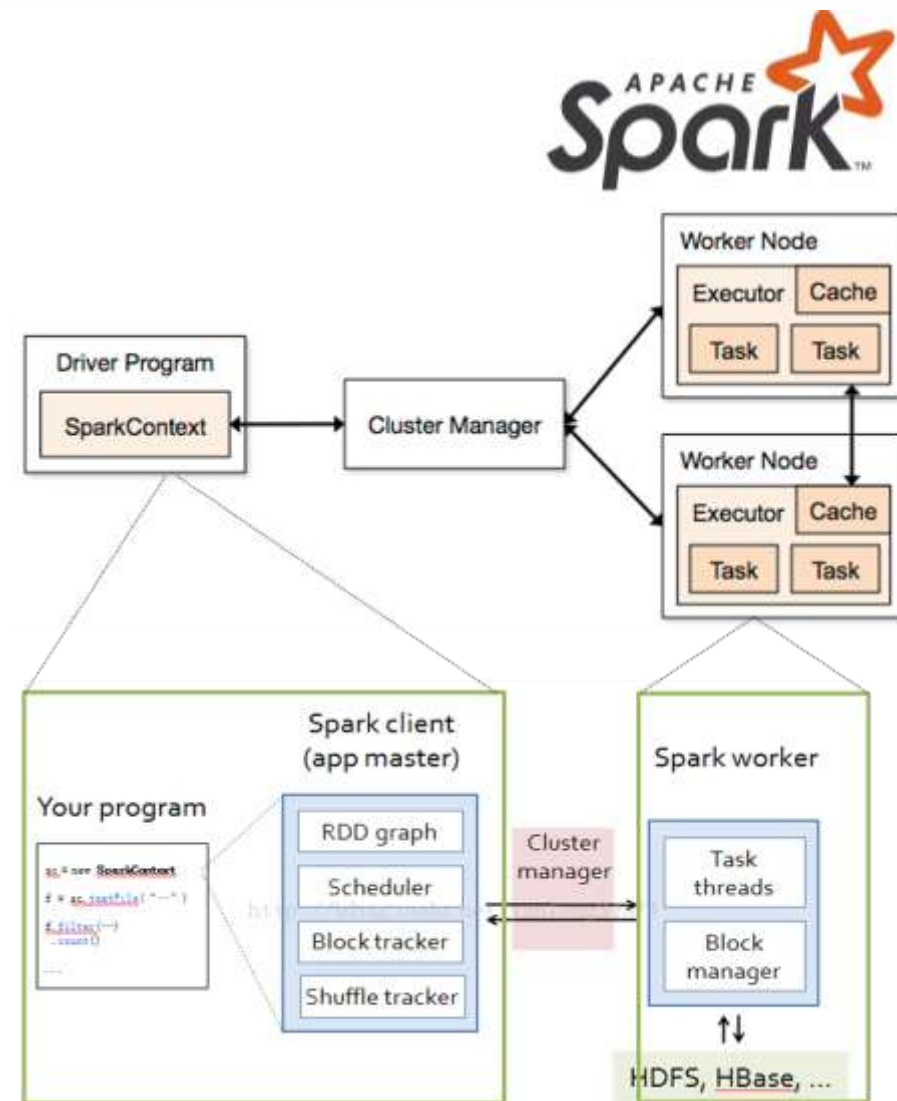
• Spark

– 简介

- UC Berkeley开源的大规模数据分析一体化引擎，类似MapReduce但使用内存计算的通用并行框架
- 缓存机制支持反复迭代计算或者多次数据共享

– 工作流程

- 驱动器向集群管理器 (YARN/Mesos) 申请运行任务所需资源，为任务分配工作节点并创建执行器
- 驱动器将用户编写的作业代码拆分成多个阶段，每个阶段创建一批任务，分配到各执行器执行
- 执行器在阶段全部任务执行后将中间结果返回驱动器或写入HDFS，驱动器再进行下一阶段调度





分布式计算服务

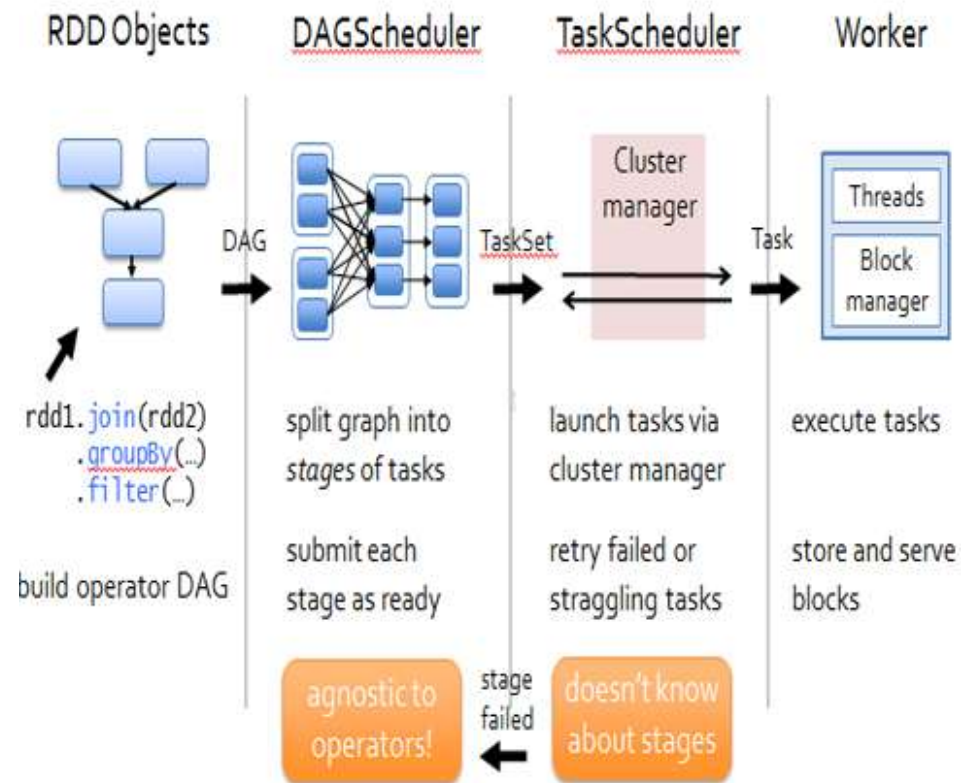
• Spark

– 调度过程

- 体现不同于MapReduce的DAG思想
- DAG调度器把一个Spark作业转换成阶段DAG, 按阶段以任务集的形式提交给任务调度器
- 任务调度器在集群的执行器运行具体任务, 重试失败或滞后的任务, 并与底层资源调度系统交互
- 弹性分布式数据集 (RDD): 一种容错并行数据结构, 允许用户保留中间结果, 控制分片来优化在内存中的位置信息



Flink: 对大规模流动数据准实时处理, 捕捉有用信息输出 (流计算)



* Matei Zaharia, Mosharaf Chowdhury, et al., "Spark: Cluster Computing with Working Sets", HotCloud 2010





分布式计算服务

• TensorFlow



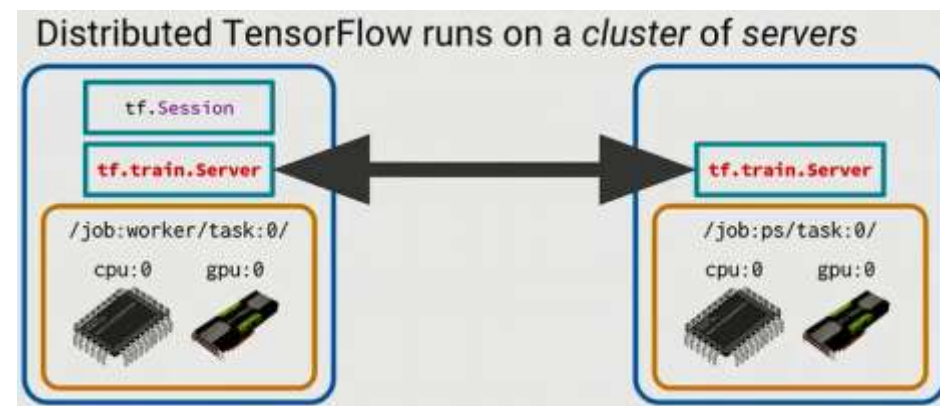
– 简介

- Google Brain开发的开源机器学习框架，致力于数据流图的自动微分和DNN计算
- **Horovod**: Uber于2017年发布的一个基于TensorFlow的高性能分布式深度学习训练框架，对常见的**NCCL**等优化库做了打包，提高了易用性

– 系统架构

- **Cluster**: 分布式系统的所有节点或者设备 (Server的集合)
- **Server**: 与Task对应，包含Master和Worker组件
- **Client**: 一个程序，创建了计算图，与Cluster交互

* Martín Abadi , Ashish Agarwal , Paul Barham, et al., "TensorFlow : Large-Scale Machine Learning on Heterogeneous Distributed Systems" , OSDI 2016





分布式机器学习框架

- PyTorch

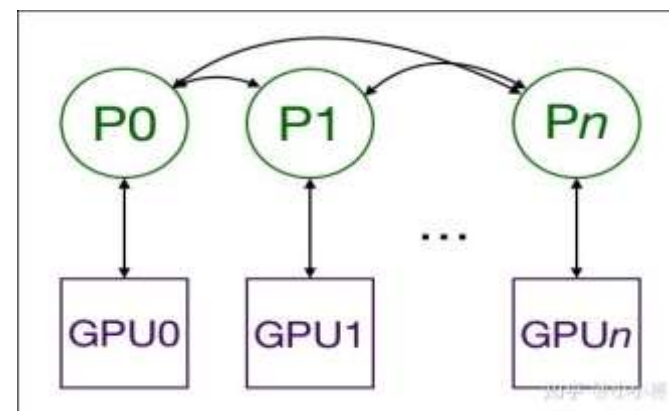


- 简介

- Facebook于2016年发布的开源深度学习框架，在主流框架中对于灵活性和易用性的平衡最好

- 训练模式

- 两种方法在多个GPU上切分模型和数据
 - DataParallel: 参数服务器 (PS) 架构，单进程多线程，在每个批处理期间将分发到每个GPU
 - DistributedDataParallel: 环规约 (Ring All-Reduce) 架构，支持模型并行，多进程，每个进程有自己的Python解释器和优化器



* Adam Paszke , Sam Gross , Francisco Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library" , NeurIPS 2019



分布式通信框架

• gRPC



– 简介

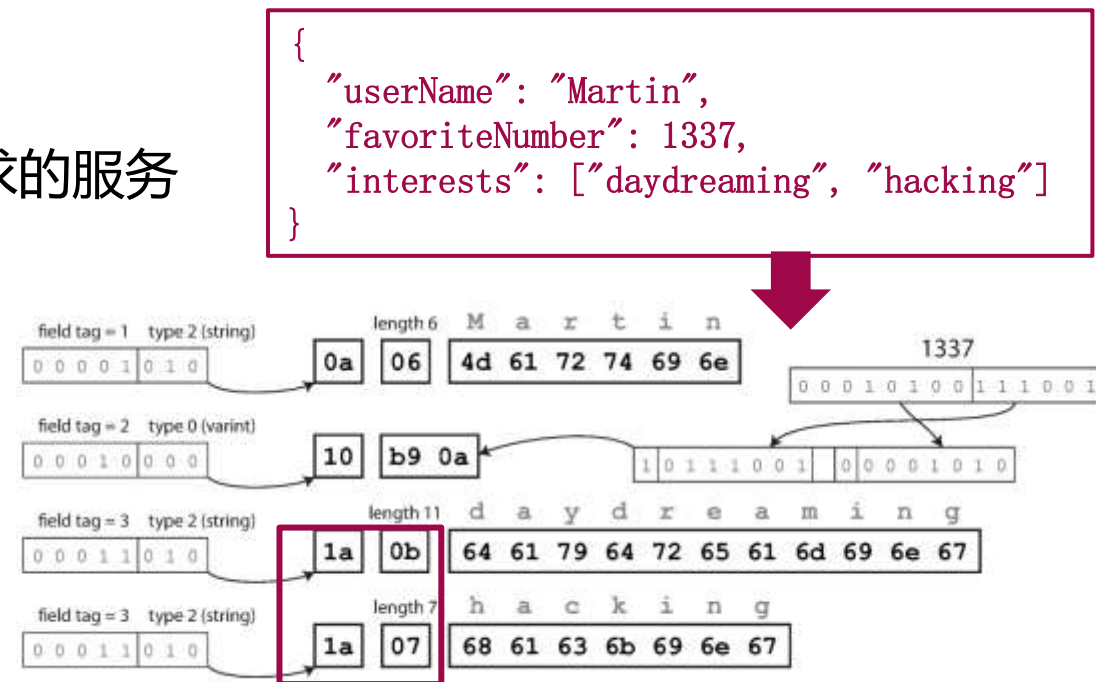
- 远程过程调用 (RPC) 广泛用于分布式节点间通信 (GFS、MapReduce、TensorFlow)
- gRPC是使用Protocol Buffers的RPC实现, 支持流 (一系列请求和响应)

– 其他RPC框架

- 编码封装可能失败的异步操作, 合并并行请求的服务
- Finagle使用Thrift, Hadoop使用Avro
- Rest.li使用JSON over HTTP (RESTful API)



跨编程语言!





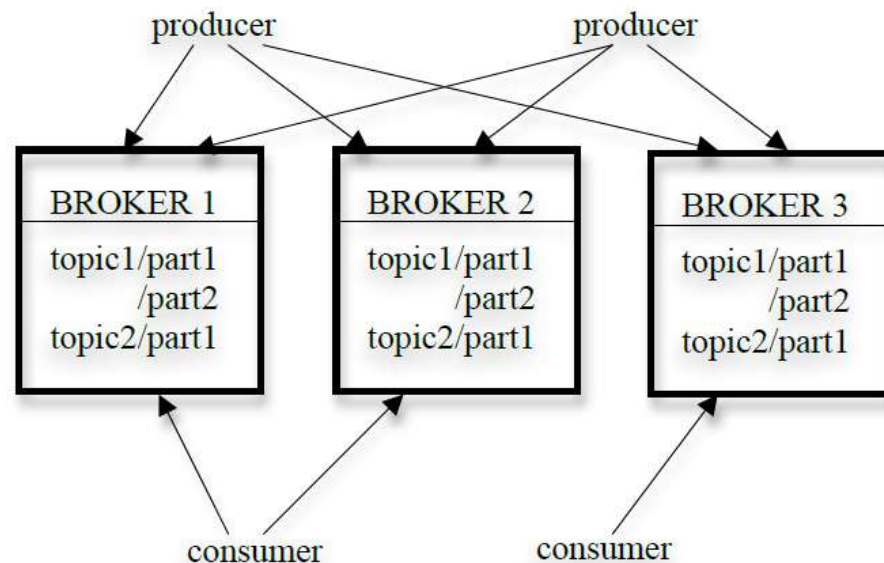
• Kafka

– 简介

- 消息中间件用于简化分布式应用间的数据传输，屏蔽底层的异构操作系统和网络平台
- Kafka是LinkedIn于2010用Scala开发的消息中间件，2012成为Apache顶级项目
- 解决海量高吞吐日志消息的处理（消息确认/原子写入不是必选项，会牺牲吞吐性能）

– 架构组件

- 生产者：推模式发布消息（访问或日志）
- 消费者：拉模式订阅 & 消费消息（应用服务）
- 代理服务器：支持水平扩展提升吞吐率
- 消息的类别称为主题（Topic），可分为多个分片（Partition）在不同代理服务器并行处理





分布式通信框架

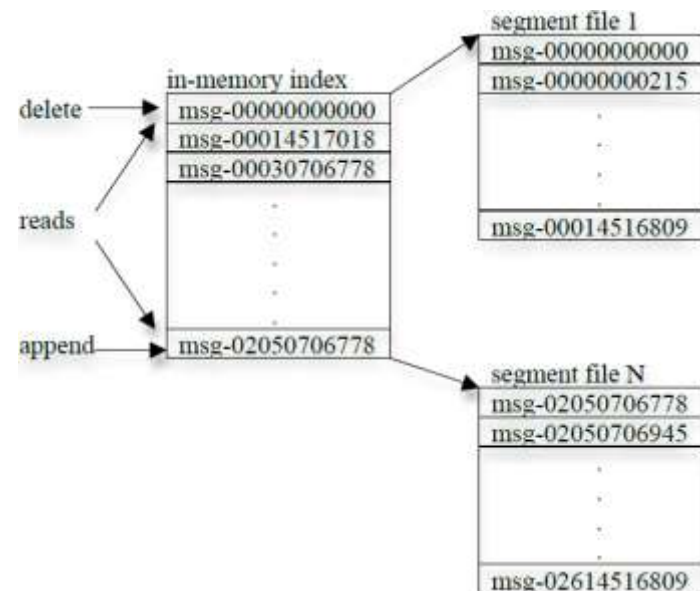
• Kafka

– 分片设计

- 每个分片对应于一个文件夹，有多个段 (Segment)，各包含消息数据和索引文件
- 代理服务器不保存消息的状态，消费者轮询时返回由生产者写入但还未消费的消息
- 消息在分片中顺序排列，通过偏移量确定位置 -> 消费者变更时新消费者可以继续消费



RocketMQ: Alibaba开发的国产中间件，具有灵活的可扩展性



* Jay Kreps, Neha Narkhebe, and Jun Rao, "Kafka: a Distributed Messaging System for Log Processing", NetDB 2011





专题作业说明

- **论文研读报告 (10分)**

- 4个主题：分布式共识、分布式系统、分布式机器学习、云计算
- 根据自己兴趣选择**任一主题两篇以上**推荐论文阅读，撰写一篇**1000字左右**阅读报告
- 重点总结论文的**核心思想**（最好能串联起两篇或多篇论文形成体系），并给出个人**心得体会**（当前研究借鉴之处、改进提升之处等），**结课日（12月23日）提交**

- **论文研讨**

- 按主题分为4个小组，在**第14周（12月9日）**开展研讨，综述主题推荐论文（或其他经典论文）的研究背景和价值、研究内容设计亮点等，**报告15分钟+研讨5分钟**
- 通过填写群问卷报名组队（先到先得），每组最多4人，加入论文报告小组**+2分**，在课堂上积极参与论文研讨**+1分**





推荐阅读论文

• 分布式共识

- 1. [SIGACT News'01] Leslie Lamport, "Paxos Made Simple" (*Paxos*简述)
- 2. [TOCS'98] Leslie Lamport, "The Part-Time Parliament" (*Paxos*)
- 3. [ATC'14] Diego Ongaro and John Ousterhout, "In Search of an Understandable Consensus Algorithm" (Extended Version) (*Raft*)
- 4. [ATC'10] Patrick Hunt, Mahadev Konar, Flavio P. Junqueira, and Benjamin Reed, "ZooKeeper: Wait-free coordination for Internet-scale systems" (*ZooKeeper*)
- 5. [LADIS'08] Benjamin Reed, and Flavio P. Junqueira, "A simple totally ordered broadcast protocol" (*ZAB*)
- 6. [OSDI'06] Mike Burrows, "The Chubby lock service for loosely-coupled distributed systems" (*Chubby*)
- 7. [TPLS'82] Leslie Lamport, Robert Shostak, Marshall Pease. "The Byzantine Generals Problem" (*拜占庭将军问题*)





• 分布式系统

- 1. **[MSST'10]** Konstantin Shvachko, Hairong Kuang, Sanjay Radia, Robert Chansler, "The Hadoop Distributed File System" (*HDFS*)
- 2. **[OSDI'06]** Sage A. Weil, Scott A. Brandt, Ethan L. Miller, et al, "Ceph: A Scalable, High-Performance Distributed File System" (*Ceph*)
- 3. **[OSDI'12]** James C. Corbett, Jeffrey Dean, Michael Epstein, et al, "Spanner: Google's Globally-Distributed Database" (*Spanner*)
- 4. **[SOSP'07]** Giuseppe DeCandia, Deniz Hastorun, Madan Jampani, et al, "Dynamo: Amazon's Highly Available Key-value Store" (*Dynamo*)
- 5. **[NSDI'12]** Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, et al., "Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing" (*Spark RDD*)
- 6. **[IEEE DEB'15]** Paris Carbone, Stephan Ewen, Seif Haridi, "Apache Flink™: Stream and Batch Processing in a Single Engine" (*Flink*)
- 7. **[NetDB'11]** Jay Kreps, Neha Narkhebe, and Jun Rao, "Kafka: a Distributed Messaging System for Log Processing" (*Kafka*)





• 分布式机器学习

- 1. **[OSDI'16]** Martín Abadi , Ashish Agarwal , Paul Barham, et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems” (*TensorFlow*)
- 2. **[arXiv'18]** Alexander Sergeev, Mike Del Balso, “Horovod: Fast and Easy Distributed Deep Learning in TensorFlow” (*Horovod*)
- 3. **[NeurIPS'19]** Adam Paszke , Sam Gross , Francisco Massa, et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library” (*PyTorch*)
- 4. **[ICML'22]** Samyam Rajbhandari, Conglong Li, et al., “DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale” (*DeepSpeed*)
- 5. **[ATC'23]** Xianyan Jia, Le Jiang, Ang Wang, et al., “Whale: Efficient Giant Model Training over Heterogeneous GPUs” (*Whale*)
- 6. **[ACL'23]** Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, et al., “Petals: Collaborative Inference and Fine-Tuning of Large Models” (*Petals*)
- 7. **[SIGCOMM'24]** Kun Qian, Yongqing Xi, Jiamin Cao, et al, “Alibaba HPN: A Data Center Network for Large Language Model Training” (*HPN*)





• 云计算

- 1. **[SOSP'11]** Brad Calder, Ju Wang, Aaron Ogus, et al, "Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency" (*Azure*)
- 2. **[SIGMOD'17]** Alexandre Verbitski, Anurag Gupta, Debanjan Saha, et al. "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases" (*Aurora*)
- 3. **[ATC'22]** Mostafa Elhemali, Niall Gallagher, Nick Gordon, et al, "Amazon DynamoDB: A Scalable, Predictably Performant, and Fully Managed NoSQL Database Service" (*DynamoDB*)
- 4. **[IMC'12]** Idilio Drago, Marco Mellia, Maurizio M. Munafò, et al, "Inside Dropbox: Understanding Personal Cloud Storage Services" (*Dropbox*)
- 5. **[EuroSys'15]** Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, et al, "Large-Scale Cluster Management at Google with Borg" (*Borg*)
- 6. **[ATC'23]** Marc Brooker, Mike Danilov, Chris Greenwood, Phil Piwonka, "On-demand Container Loading in AWS Lambda" (*Lambda*)
- 7. **[SEC'18]** Ying Xiong, Yulin Sun, Li Xing, Ying Huang, "Extend Cloud to Edge with KubeEdge" (*KubeEdge*)





谢谢

